

FITS HRMS — Code Walkthrough

What to show and how to show it (Easy English)

Use this right after the slides, before the live website walkthrough. Open each file in VS Code, point to the part shown, and say the italic line.

TIP: Increase your editor font size before you start, so everyone can read it. Keep it short — show the file, say one or two lines, move on. Do NOT read code line by line.

STEP 1

Show the project structure

OPEN: the folder tree in VS Code (left panel)

POINT TO: the list of app folders — base, employee, recruitment, leave, attendance, payroll, pms, onboarding, omani_compliance, ui.

“Let me first show how the project is organized. The whole system is split into small parts called apps, and each app does one job. For example, the recruitment app handles hiring, the payroll app handles salary, and the omani_compliance app handles the special Oman rules. The new screens I worked on are in the ui app. Keeping the code in separate apps like this makes it clean and easy to manage.”

STEP 2

Show the data models (the database design)

OPEN: recruitment/models.py

POINT TO: the class names — Candidate, Recruitment, InterviewSchedule, OfferLetter. (Use the outline / Ctrl+F to jump to 'class Candidate'.)

“This file describes the data we store. Each class here becomes a table in the database. For example, Candidate stores the people who apply, Recruitment stores the job openings, and OfferLetter stores the final offers. I don't write any SQL by hand — Django turns these classes into database tables for me.”

IF ASKED why classes and not SQL:

Django's ORM lets us describe data as Python classes; it creates and updates the tables automatically through migrations.

STEP 3

Show the approval engine (the hiring rules)

OPEN: `recruitment/approvals/engine.py`

POINT TO: the functions that route approvals (e.g. `route_offer` / the step ordering logic).

"This is the brain of the approval flow. When a request needs approval, this code decides who approves it and in what order. The important rule is that approvals happen one by one — only the first person's turn is active, and the next person can only act after the one before them signs. This gives a clear record of who approved what and when."

STEP 4

Show the AI CV screening (my main work)

OPEN: `recruitment/cv_screening_ai.py`

POINT TO: the docstring at the very top — the three-tier fallback chain.

"This is the AI feature I worked on the most. When a CV comes in, this code sends it to an AI model with a prompt, and the AI reads the CV and gives a score for each candidate. The clever part is at the top of the file: I built a three-step backup system. First it tries Groq, which runs a Llama model and is very fast. If that is not available, it falls back to a second model called DeepSeek through OpenRouter. And if no AI is available at all, it falls back to simple rule-based reading. So the feature keeps working even if one AI service is down."

ON SCREEN (read this from the file):

- 1. Groq — Llama 3.1 (main, fast)
- 2. OpenRouter — DeepSeek (backup)
- 3. Rule-based reading (last resort, needs no AI key)

STEP 5

Show the AI prompt (how we ask the AI)

OPEN: `recruitment/cv_screening_ai.py` (the `_build_prompt` function)

POINT TO: the prompt text inside `_build_prompt` — the 8 scoring criteria.

"This is the message we send to the AI. I wrote it to act like an HR expert and score the candidate on eight clear points — like work experience, qualifications, and technical skills — each out of ten. Writing a clear prompt like this is the most important part of getting good answers from the AI. This is also where I tried different models — I started with Llama, then moved to Groq for speed, and kept testing to find the best results."

STEP 6

Show the new UI views

OPEN: `ui/views.py`

POINT TO: a couple of view functions, e.g. `candidates()` and `candidate_delete()`.

"Finally, this file holds the screens for the new interface I built. Each function here prepares the data and shows one page — for example, the candidates list and the candidate delete action. This is the part that connects the database and the AI to what the user actually sees on the website."

OPTIONAL (shows real problem-solving):

You can mention: 'Deleting a candidate first failed because of a protected link to proposals; I fixed it by removing the linked records safely before deleting.'

REFERENCE — Every function in `cv_screening_ai.py`

If anyone asks about a specific function, here is what each one does, top to bottom. (You don't need to read all of these aloud — use them to answer questions.)

`_build_prompt(cv_content, job_requirements)`

Builds the instruction message we send to the AI. It tells the AI to act like an HR expert, read the CV, score the candidate on 8 points (each out of 10), and return the answer as clean JSON.

`_analyze_with_groq(cv_content, job_requirements)`

Tier 1 (main). Sends the CV to Groq's Llama 3.1 model and returns the AI's JSON result. Returns nothing if there is no API key or the call fails — so the next tier can try.

`_analyze_with_openrouter(cv_content, job_requirements)`

Tier 2 (backup). If Groq is not available, this sends the same CV to the DeepSeek model through OpenRouter and returns its result.

`_analyze_with_regex(cv_content, job_requirements)`

Tier 3 (last resort). If no AI is available at all, this uses simple rule-based reading to pull out basic details — so the feature never fully breaks.

`_analyze_cv(cv_content, job_requirements)`

The decision-maker. It tries Groq first, then OpenRouter, then the rule-based method, and returns the first one that works — and notes which model was used.

`_extract_text(file_path)`

Opens the uploaded file — PDF, Word, or text — and pulls out the plain text from it, so the AI has something to read. This is the 'extract the CV' step.

`class CVScreeningAI`

An older helper class kept so existing code still works. It wraps the functions above (extract text, analyze, rank, and auto-shortlist).

`CVScreeningAI.extract_cv_content(...)`

Gets the plain text out of a CV file (uses `_extract_text`).

`CVScreeningAI.analyze_cv_with_deepseek(...)`

Runs the full AI analysis chain on a CV (Groq → OpenRouter → rules).

`CVScreeningAI.rank_candidates(...)`

Sorts a list of candidates by their match score, best first.

CVScreeningAI.auto_shortlist(...)

Screens many candidates at once and marks the ones above the score limit as shortlisted.

screen_candidate_cv(candidate_id, job_requirements)

The MAIN function. It extracts the resume text, runs the AI, works out the final score and the recommendation (auto-shortlist / interview / reject), and saves everything to the candidate's screening profile. This is the heart of CV screening.

_job_requirements_for(candidate)

Builds the 'what the job needs' text from the job linked to the candidate, so the AI knows what to compare the CV against.

trigger_screening_async(candidate_id)

Runs the screening in the background (a separate thread) so the website does not freeze while the AI is working. This is what the upload pages actually call.

CLOSING — TRANSITION TO THE WEBSITE

“So that is the code behind the system — the data, the approval rules, and the AI. Now let me open the website and show you how it all works together, by following one candidate from CV upload all the way to a signed offer.”